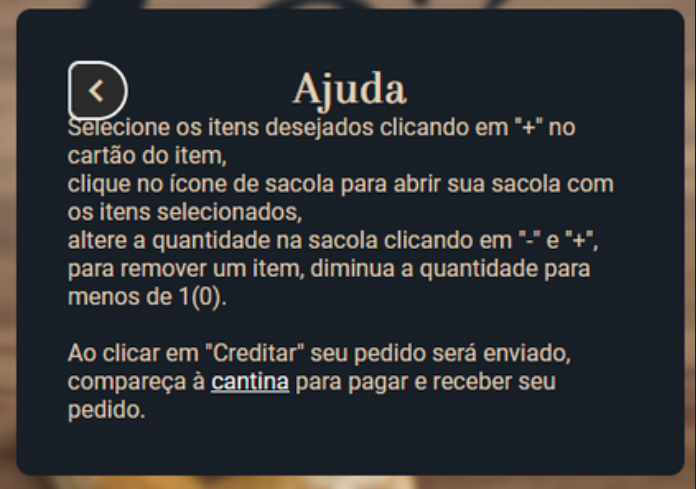
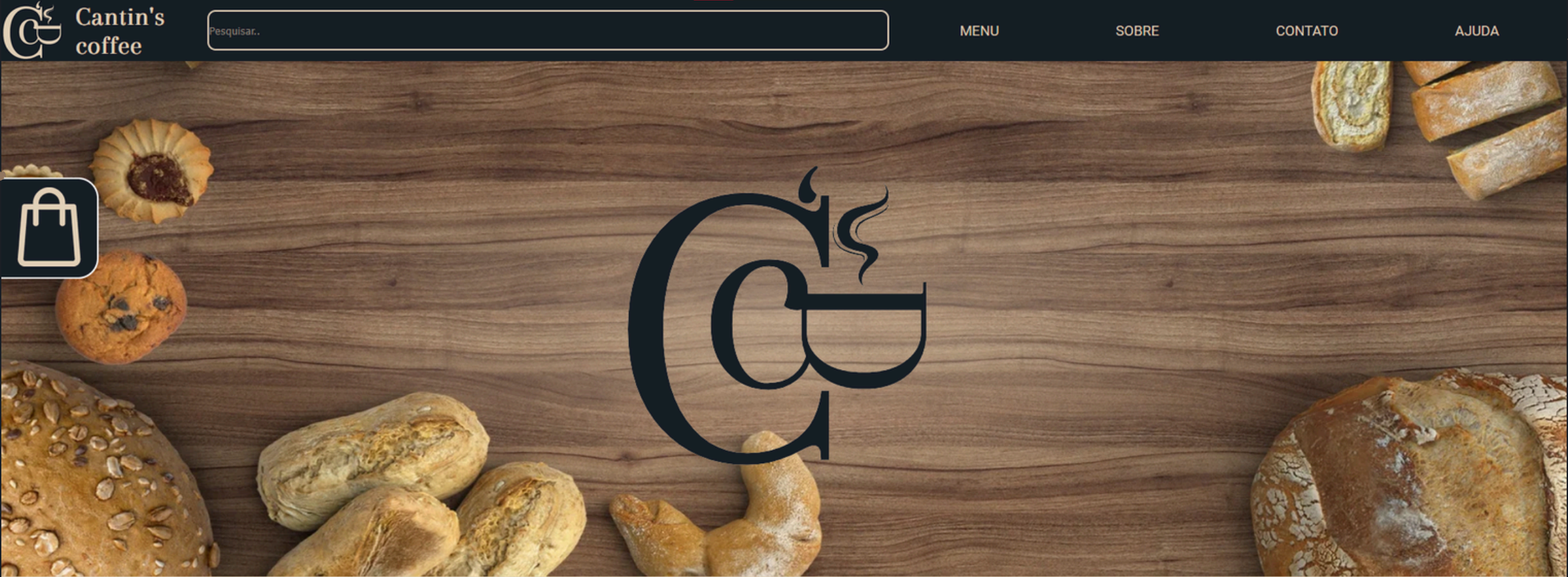


Cantin's coffee

Site pessoal no github pages

Site para uma cafeteria, extremamente simplificado, roda em apenas uma página usando pop-ups



Teste

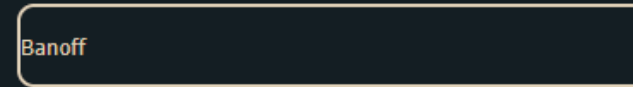
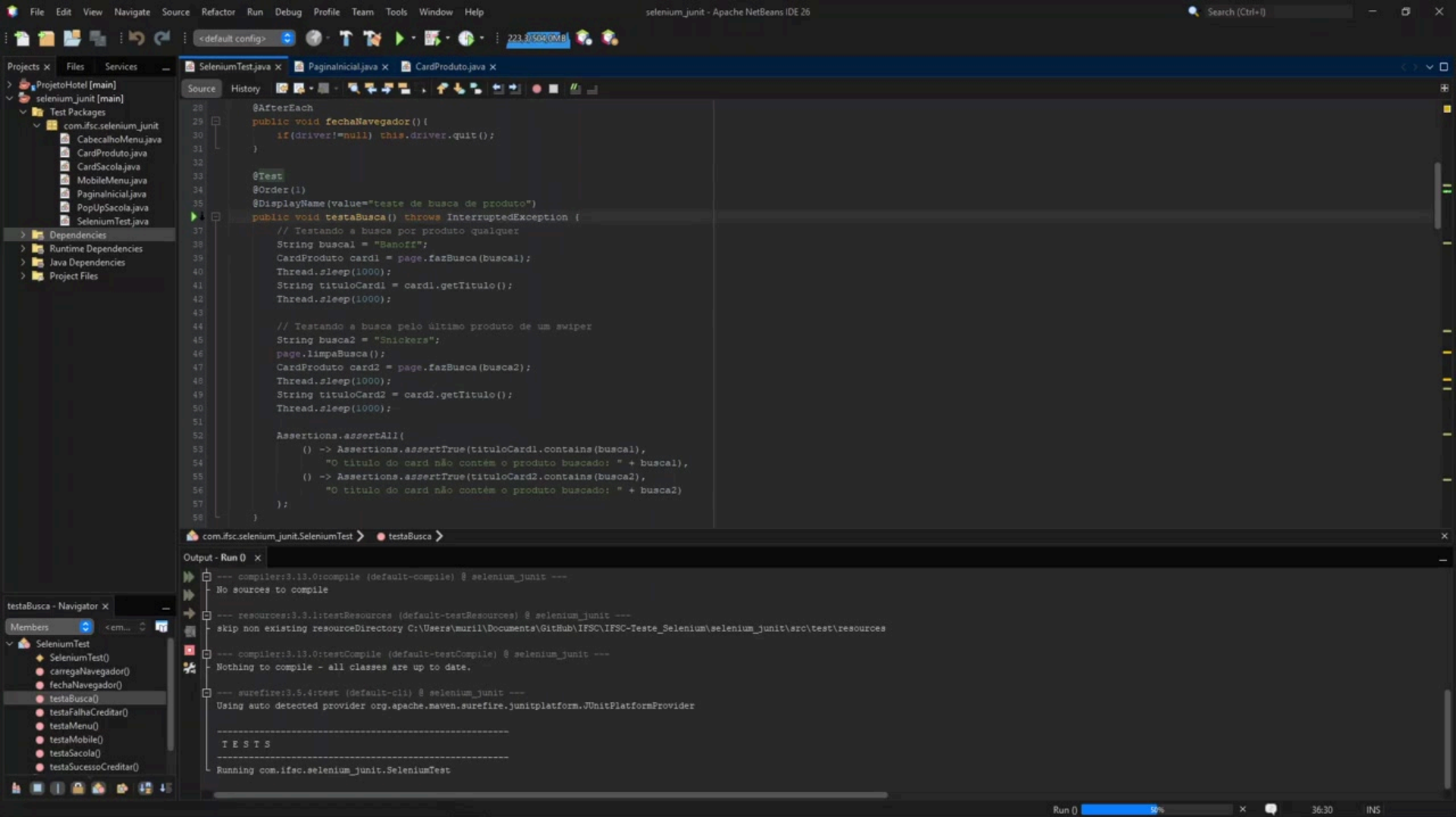
#1

Descrição do teste (funcionalidade testada):	Busca de produto no cardápio.
Iteração com o sistema:	Estando na página inicial, click na barra de pesquisa. Preenchimento do campo com nome do item. Pesquisa realizada sem necessidade de submissão, card do produto é destacado, pagina move-se para o local do produto automaticamente.
Resultado esperado	Produto com nome pesquisado em destaque.
Resultado obtido	Pagina moveu-se para o local do produto e o destacou.
Resutlado Teste	Sucesso.
Observação	Produto em destaque recebe a classe <u>swiper-slide-active</u> . Caso o produto seja o último da lista o <u>swiper</u> não consegue alcança-lo e outro item recebe a classe ativa. Funciona apenas buscando o produto com classe buscado.

```
@Test
@Order(1)
@DisplayName(value="teste de busca de produto")
public void testaBusca() throws InterruptedException {
    // Testando a busca por produto qualquer
    String buscal = "Banoff";
    CardProduto card1 = page.fazBusca(buscal);
    Thread.sleep(1000);
    String tituloCard1 = card1.getTitulo();
    Thread.sleep(1000);

    // Testando a busca pelo último produto de um swiper
    String busca2 = "Snickers";
    page.limpaBusca();
    CardProduto card2 = page.fazBusca(busca2);
    Thread.sleep(1000);
    String tituloCard2 = card2.getTitulo();
    Thread.sleep(1000);

    Assertions.assertAll(
        () -> Assertions.assertTrue(tituloCard1.contains(buscal),
            "O título do card não contém o produto buscado: " + buscal),
        () -> Assertions.assertTrue(tituloCard2.contains(busca2),
            "O título do card não contém o produto buscado: " + busca2)
    );
}
```



```
<div class="card swiper-slide swiper-slide-active buscado">
  <div class="image-content">
  <div class="card-content">
    <div class="produto">
      <h2 class="nome">Banoff</h2>
      <p class="descricao">
        Sobremesa de biscoito, camada de doce de leite e banana.
        Contém glúten e lactose.
      </p>
    </div>
    <div class="valores">
    </div>
  </div>
```

Page Objects

PaginaInicial: clica no ícone da sacola, envia String para a barra de pesquisa e retorna opcoes clicáveis do menu

```
public class PaginaInicial {
    private WebDriver driver;
    private By campoBusca = By.className("barra-pesquisa");
    private By labelMenu = By.tagName("menu");

    public PaginaInicial(WebDriver driver){
        this.driver = driver;
    }

    public void navegaHome(){
        this.driver.get("http://muriloschneiders.github.io/SENAC-WEB-cantin-sCof");
    }

    public void toggleSacola(){
        WebElement btnSacola = driver.findElement(By.className("icone-sacola"));
        btnSacola.click();
    }

    public CardProduto fazBusca(String valor){
        //Busca apenas move o campo ativo para o item com nome contendo termo
        //pesquisado, sem precisar de enter
        WebElement campoSearch = driver.findElement(campoBusca);
        campoSearch.sendKeys(valor);
        return new CardProduto(this.driver);
    }

    public void limpaBusca(){
        WebElement campoSearch = driver.findElement(campoBusca);
        campoSearch.clear();
    }

    public CabecalhoMenu getMenu(){
        WebElement menuVisivel=driver.findElement(labelMenu);
        //Menu mobile invisível com tela cheia no computador
        for (WebElement menu : driver.findElements(labelMenu)) {
            if (menu.isDisplayed()) {
                menuVisivel = menu;//Atribui o menu apenas se ele estiver visível
                break;//Sai do loop se encontrar um elemento visível
            }
        }

        return new CabecalhoMenu(this.driver, menuVisivel);
    }
}
```

CardProduto: armazena as informações do card de um produto dentro de um slider na pagina inicial.

```
public class CardProduto{
    private WebDriver driver;
    //Card Ativo dentro do swiper slide
    private By labelCardAtivo= By.className("swiper-slide-active");
    //Card buscado pela barra de pesquisa
    private By labelCardBuscado= By.className("buscado");
    private final WebElement cardBuscado;

    private final String titulo;
    private final Double preco;
    private final WebElement botaoAdd;

    public CardProduto(WebDriver driver) {
        this.driver= driver;
        cardBuscado = this.driver.findElement(labelCardBuscado);

        this.titulo = cardBuscado.findElement(By.tagName("h2")).getText();
        this.preco = Double.valueOf(cardBuscado.findElement(
            By.className("menu-card-preco")).getText().replace(", ", "."));
        this.botaoAdd = cardBuscado.findElement(By.className("add-button"));
    }

    public String getTitulo(){
        return titulo;
    }

    public double getPreco(){
        return preco;
    }

    public void addSacola(){
        botaoAdd.click();
    }

    public WebElement slider(){//slider/carrossel onde o item esta
        return cardBuscado.findElement(
            By.xpath("ancestor::div[contains(@class, 'slide-content')]"));
    }
}
```


Teste

#2

Descrição do teste (funcionalidade testada):	Adicionar produto na sacola.
Iteração com o sistema:	Click no botão + de algum produto, 2 ou mais clicks em outro produto, conferir a sacola.
Resultado esperado	Produto deve estar na sacola com a quantidade equivalente a quantidade de vezes que o botão + foi clicado.
Resultado obtido	Produtos encontrados na sacola com informações condizentes.
Resutlado Teste	Sucesso.
Observação	Sacola é um pop-up.

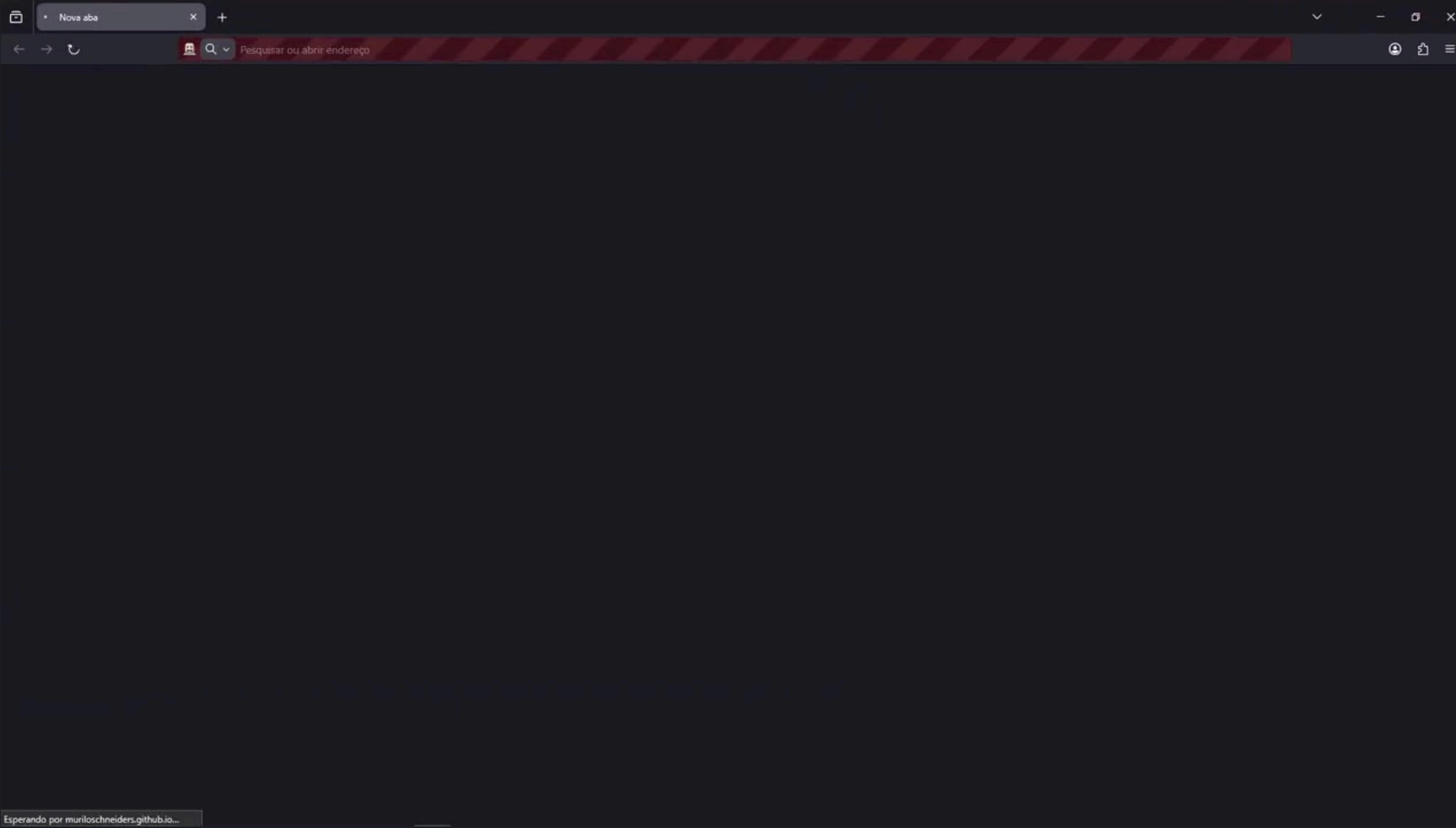
```
public void testaSacola() throws InterruptedException{
    //Adicionar produto à sacola
    String nomeProdutol = "RedBull";
    CardProduto cardl = page.fazBusca (nomeProdutol);
    Thread.sleep(2000);
    cardl.addSacola (); //1X

    page.limpaBusca ();

    String nomeProduto2 = "Baly";
    CardProduto card2 = page.fazBusca (nomeProduto2);
    Thread.sleep(2000);
    card2.addSacola ();
    card2.addSacola (); //2X

    //Verificar se o produto está na sacola
    page.toggleSacola ();
    PopUpSacola sacola = new PopUpSacola(driver);
    List<CardSacola> itens = sacola.getItens (); //Armazena os itens uma vez

    //Verificações
    for (CardSacola item : itens){ //Para cada item na sacola
        if(item.getNome().equals(nomeProdutol)){
            Assertions.assertTrue(item.getQuantidade()==1);
            Assertions.assertTrue(item.getPreco()==cardl.getPreco());
        }
        if(item.getNome().equals(nomeProduto2)){
            //2 baly pois foi clicado "+" 2 vezes
            Assertions.assertTrue(item.getQuantidade()==2);
            Assertions.assertTrue(item.getPreco()==card2.getPreco());
        }
        Assertions.assertTrue(
            item.getNome().equals(nomeProdutol) ||
            item.getNome().equals(nomeProduto2),
            "Produto " + item.getNome() + " não está na sacola."
        );
    }
}
```



RedBull

das



Red Bull
Energético

R\$14,00

+

adicionar

< Sacola de Murilo Schneider



Red Bull
Energético

R\$14,00

1

+



Baly
lata 473ml
Energético

R\$10,00

2

+

Ao pressionar 'Creditar', seu pedido será enviado e você poderá pagar e recebê-lo no [local](#).

Creditar R\$34,00

Page Objects

PopUpSacola: Interage com o pop-up da sacola e retorna uma lista de CardSacola dos produtos presentes na sacola.

```
public class PopUpSacola {
    private WebDriver driver;
    private final By labelLista= By.tagName("ul");//lista dos itens(li) na sacola
    private final By labelFinalizar = By.className("finalizar");
    private final By labelNome = By.className("input-cliente");

    public PopUpSacola(WebDriver driver) {
        this.driver= driver;
    }

    //Preenche campo nome
    public void preencheNome(String nome){
        WebElement inputNome = driver.findElement(labelNome);
        inputNome.sendKeys(nome);
    }

    //Retornar lista dos itens(CardSacola)
    public List<CardSacola> getItens() {
        List<CardSacola> itens = new ArrayList<>();
        //Armazena o <ul>
        WebElement list = driver.findElement(labelLista);
        //Armazena todos os <li>
        List<WebElement> itensSacola = list.findElements(By.tagName("li"));

        for (WebElement item : itensSacola){
            //Busca as informações de cada item na sacola
            String itemNome = item.findElement(By.className("cart-title")).getText();
            double itemPreco = Double.parseDouble(item.findElement(
                By.className("item-preco")).getText().replace(",","."));
        };
        int itemQuantidade = Integer.parseInt(item.findElement(
            By.className("quantidade")
        ).getText());

        //Cria um objeto CardSacola para cada item
        CardSacola card = new CardSacola(driver, itemNome, itemPreco, itemQuantidade);
        itens.add(card);
    }

    return itens;
}

//Finaliza compra
public void creditar(){
    WebElement btnFinalizar = driver.findElement(labelFinalizar);
    btnFinalizar.click();
}
}
```

CardSacola: armazena e retorna as informações de um card na sacola.

```
public class CardSacola {
    private WebDriver driver;
    private String nome;
    private double preco;
    private int quantidade;

    public CardSacola(WebDriver driver, String nome, double preco, int quantidade){
        this.driver = driver;
        this.nome=nome;
        this.preco=preco;
        this.quantidade=quantidade;
    }

    public String getNome() {
        return nome;
    }

    public double getPreco() {
        return preco;
    }

    public int getQuantidade() {
        return quantidade;
    }
}
```

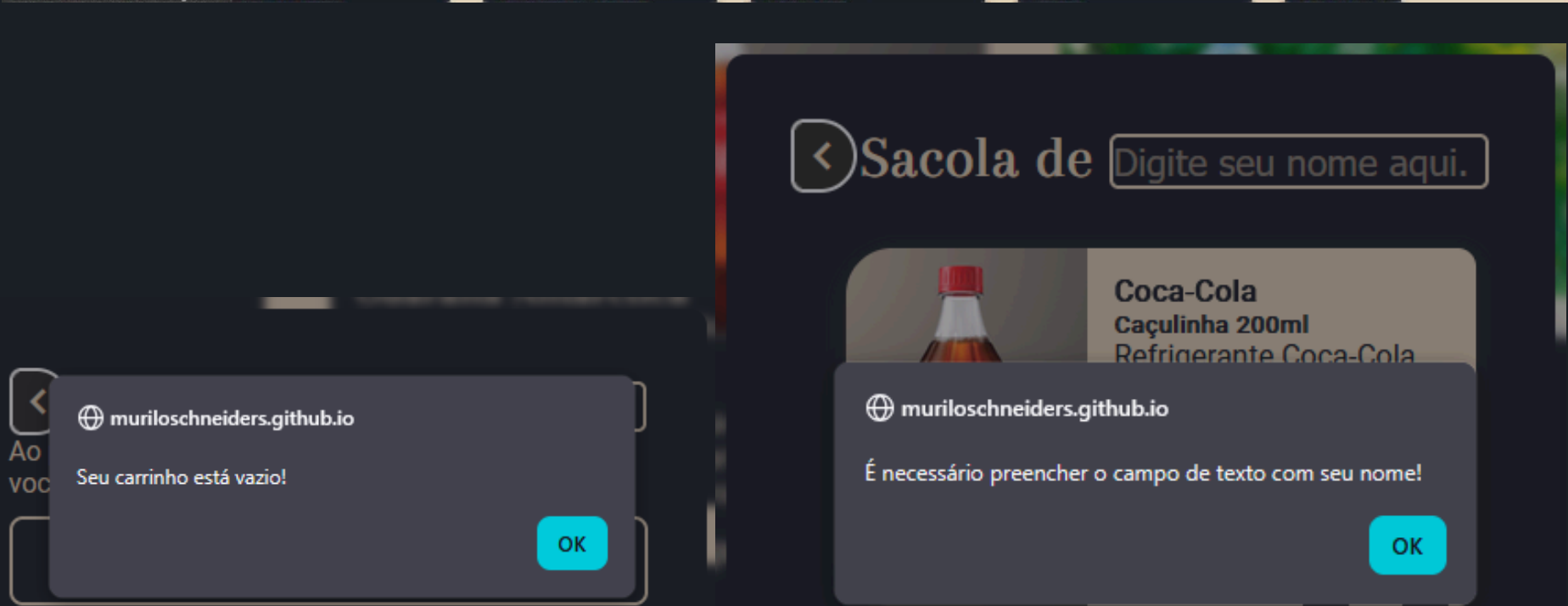
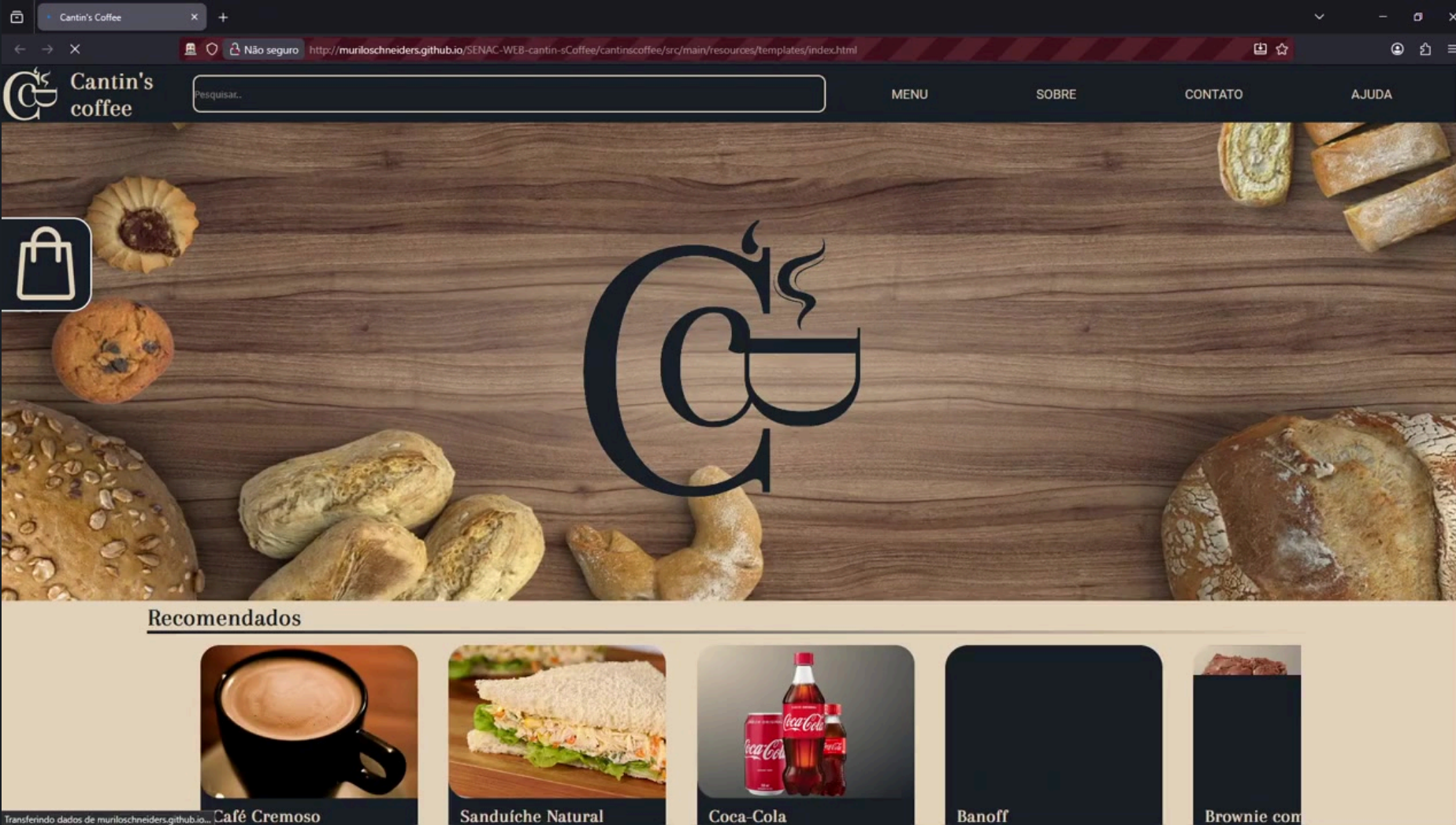

Descrição do teste (funcionalidade testada):	Creditar sem informações adequadas.
Iteração com o sistema:	Abrir a sacola e tentar creditar com ela vazia ou sem informar o nome.
Resultado esperado	Mensagem de erro informando o que é necessário.
Resultado obtido	Alertas “Seu carrinho está vazio!” e “É necessário preencher o campo de texto com seu nome!” emitidos corretamente.
Resutlado Teste	Sucesso.
Observação	Sacola é um pop-up. Alerta de sacola vazia deveria dizer “Sacola”.

```

public void testaFalhaCreditar() throws InterruptedException{
    //Sacola vazia
    page.toggleSacola();
    PopUpSacola sacola = new PopUpSacola(driver);
    sacola.creditar();
    Alert alert = driver.switchTo().alert();
    Thread.sleep(1000);
    Assertions.assertEquals(alert.getText(), "Seu carrinho está vazio!");
    alert.accept();

    //Faltando nome
    page.toggleSacola();
    Thread.sleep(500);
    CardProduto card1 = page.fazBusca("Coca");
    Thread.sleep(1000);
    card1.addSacola();

    page.toggleSacola();
    sacola.creditar();
    Thread.sleep(1000);
    Assertions.assertEquals(alert.getText(),
        "É necessário preencher o campo de texto com seu nome!");
    alert.accept();
}
    
```



Ao pressionar 'Creditar', seu pedido será enviado e você poderá pagar e recebê-lo no local.

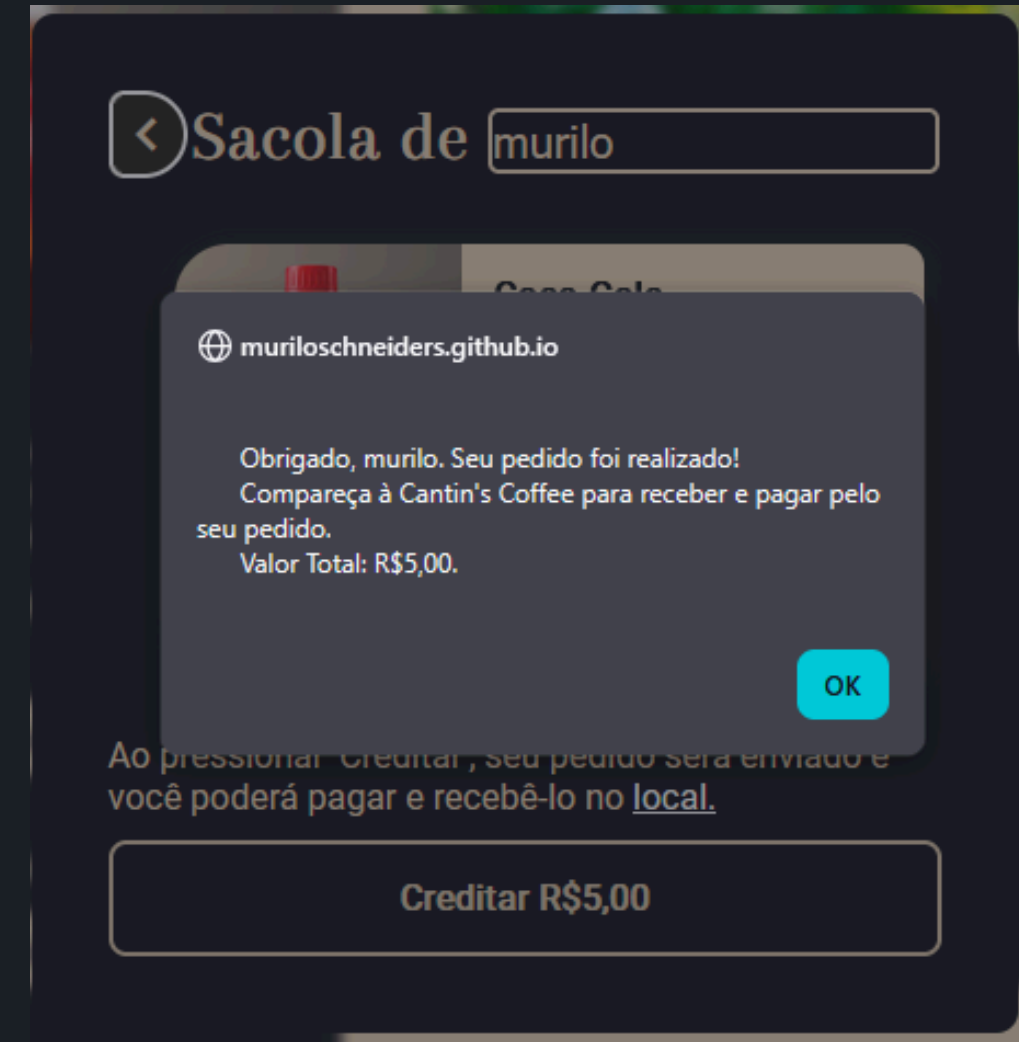
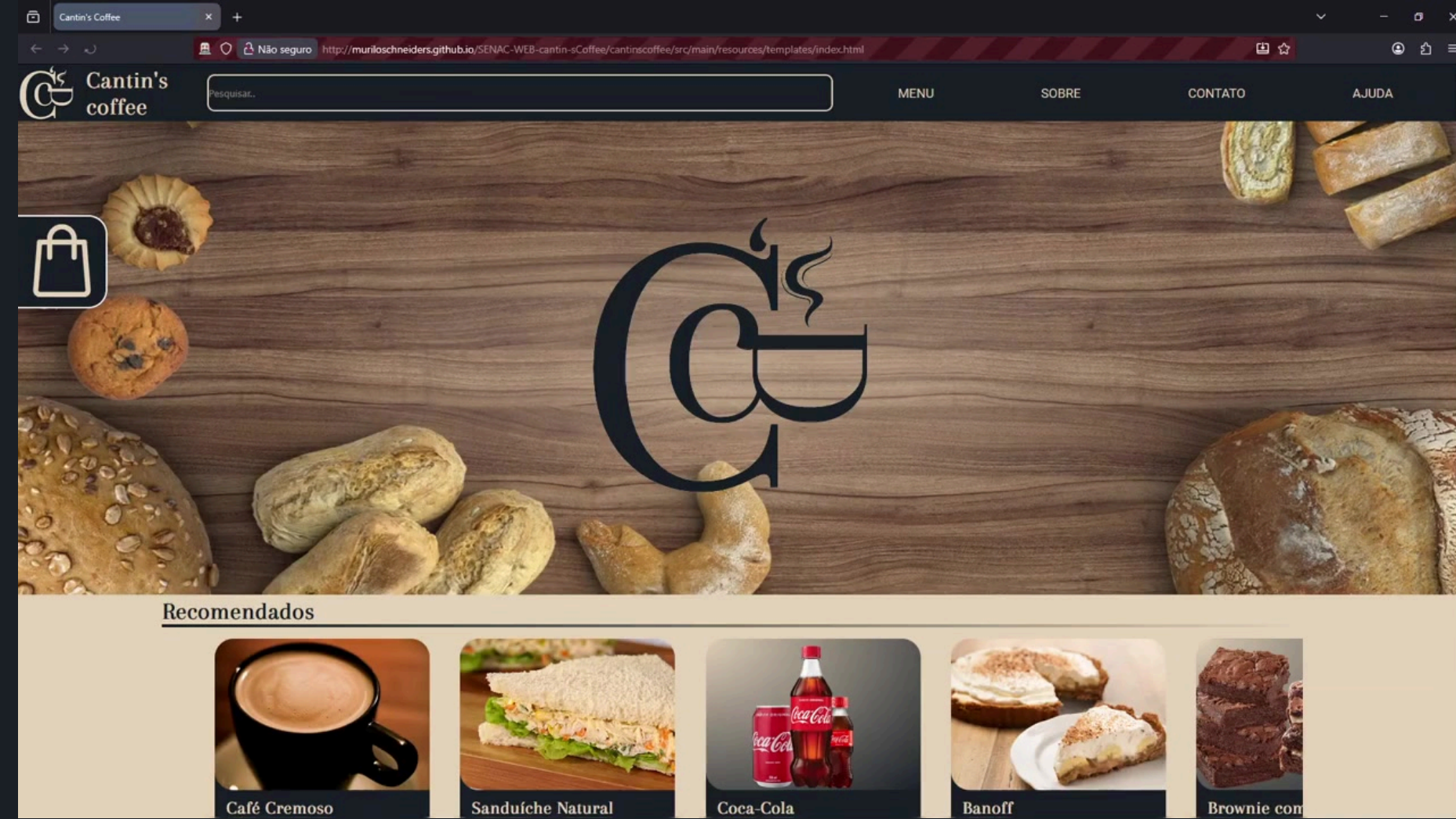
Creditar R\$5,00

Teste

#4

Descrição do teste (funcionalidade testada):	Creditar corretamente.
Iteração com o sistema:	Adicionar produtos a sacola, abrir sacola, inserir nome e click em creditar.
Resultado esperado	Mensagem de sucesso.
Resultado obtido	Alerta “Obrigado, <Nome inserido>. Seu pedido foi realizado! Compareça à <u>Cantin's Coffee</u> para receber e pagar pelo seu pedido. Valor Total: R\$<Valor dos itens>.” emitido corretamente
Resutlado Teste	Sucesso
Observação	Sacola é um pop-up.

```
public void testaSucessoCreditar() throws InterruptedException{  
    //Preencher sacola  
    CardProduto card1 = page.fazBusca("Coca");  
    Thread.sleep(1000);  
    card1.addSacola();  
  
    //Preencher nome  
    page.toggleSacola();  
    PopUpSacola sacola = new PopUpSacola(driver);  
    String nome = "murilo";  
    sacola.preencheNome(nome);  
    sacola.creditar();  
    Alert alert = driver.switchTo().alert();  
  
    Assertions.assertEquals(alert.getText().trim(), ("  Obrigado, "+nome+  
        ". Seu pedido foi realizado!\n" +  
        "Compareça à Cantin's Coffee para receber e pagar pelo seu pedido.\n" +  
        "Valor Total: R$"+String.format("%.2f",card1.getPreco())+".").trim()));  
}
```



Teste

#5

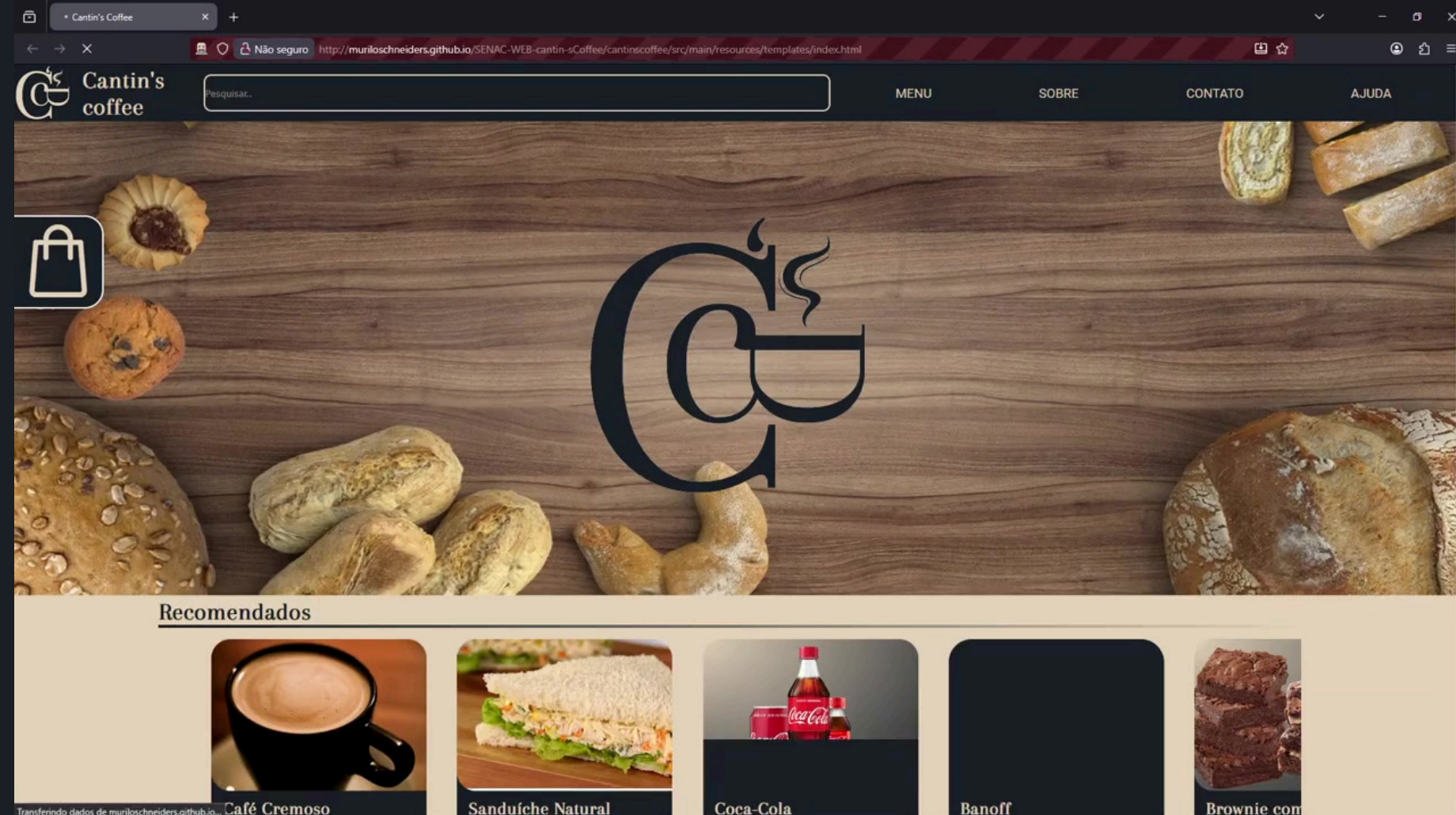
Descrição do teste (funcionalidade testada):	Testar opções do cabeçalho.
Iteração com o sistema:	Click em cada opção do cabeçalho (MENU, SOBRE, CONTATO e AJUDA).
Resultado esperado	Página deve mover-se/exibir o menu, seção de <u>sobre</u> , <u>informações</u> de contato e ajuda.
Resultado obtido	Menu levou ao cardápio, sobre e contato levaram ao rodapé contendo as informações e ajuda abriu um pop-up explicando como utilizar o site.
Resutlado Teste	Sucesso.
Observação	SOBRE e CONTATO estão juntos no rodapé, AJUDA é um pop-up.

MENU

SOBRE

CONTATO

AJUDA



```
public void testaMenu() throws InterruptedException{
    CabecalhoMenu menu = page.getMenu();
    menu.clickMenu();
    Thread.sleep(1000);
    Assertions.assertTrue(driver.getCurrentUrl().contains("#menu"), "A URL não contém '#menu'.");

    menu.clickSobre();
    Thread.sleep(1000);
    Assertions.assertTrue(driver.getCurrentUrl().contains("#sobre"), "A URL não contém '#sobre'.");

    menu.clickContato();
    Thread.sleep(1000);
    Assertions.assertTrue(driver.getCurrentUrl().contains("#faleconosco"), "A URL não contém '#faleconosco'.");

    Assertions.assertTrue(menu.clickAjuda().getAttribute("class").contains("open"), "O pop up ajuda não esta aberto.");
}
```



Ajuda

Selecione os itens desejados clicando em "+" no cartão do item, clique no ícone de sacola para abrir sua sacola com os itens selecionados, altere a quantidade na sacola clicando em "-" e "+", para remover um item, diminua a quantidade para menos de 1(0).

Ao clicar em "Creditar" seu pedido será enviado, compareça à cantina para pagar e receber seu pedido.

Page Objects

CabecalhoMenu: armazena os WebElement de cada opcao do menu e permite clica-los, também verifica se o menu esta normal ou vertical(mobile).

```
public class CabecalhoMenu {
    private WebDriver driver;
    private List<WebElement> opcoes_menu;
    private WebElement btnMobileMenu;

    public CabecalhoMenu(WebDriver driver, WebElement nav_list) {
        this.driver = driver;
        this.opcoes_menu = nav_list.findElements(By.className("nav-link"));
        this.btnMobileMenu = driver.findElement(By.className("mobile-menu-icon"));
    }

    public void clickMenu() {
        opcoes_menu.get(0).click();
    }

    public void clickSobre() {
        opcoes_menu.get(1).click();
    }

    public void clickContato() {
        opcoes_menu.get(2).click();
    }

    public WebElement clickAjuda() {
        opcoes_menu.get(3).click();
        return driver.findElement(By.className("pop-ajuda"));
    }

    public boolean checkDisplayMobileMenu() {
        return btnMobileMenu.isDisplayed();
    }
}
```

Pagina Inicial retorna o CabecalhoMenu visível (horizontal ou vertical):

```
public CabecalhoMenu getMenu() {
    WebElement menuVisivel=driver.findElement(labelMenu);
    //Menu mobile invisivel com tela cheia no computador
    for (WebElement menu : driver.findElements(labelMenu)) {
        if (menu.isDisplayed()) {
            menuVisivel = menu;//Atribui o menu apenas se ele estiver visível
            break;//Sai do loop se encontrar um elemento visível
        }
    }

    return new CabecalhoMenu(this.driver, menuVisivel);
}
```


Teste

#6

Descrição do teste (funcionalidade testada):	Redimensionar tela para celular.
Iteração com o sistema:	Redimensionar a tela para tamanho de celular
Resultado esperado	Menu deve virar hambúrguer e cardápio deve ficar vertical.
Resultado obtido	Com menos de 760px horizontais o Menu passa para dentro de um ícone de hamburger e o cardápio fica vertical.
Resutlado Teste	Sucesso.
Observação	Ícone de fechar menu vertical ausente.

```
@Test
@Order(6)
@DisplayName("testa redimensionamento da tela")
public void testaMobile() throws InterruptedException{
    driver.manage().window().setSize(new Dimension(759, 1000));
    //Menu deve virar hambúrguer.
    CabecalhoMenu menu = page.getMenu();
    Assertions.assertTrue(menu.checkDisplayMobileMenu(), "mobile-menu-icon nao esta visivel.");
    //cardápio deve ficar vertical
    Assertions.assertTrue(page.fazBusca("Brownie").slider().getAttribute("class").contains("swiper-vertical"));
}
```

